



Data Pipelines | HW 01

Выполнил: Васюков Александр, avvasiukov@edu.hse.ru

Github: <https://github.com/vasyukov1/hse-data-pipelines/tree/main/hw01>

Контекст

“Система анализа состояния и эффективности футболистов” — система, которая собирает данные об игроке.

На поле важно знать, как он двигается, сколько энергии тратит и как работает его сердце в реальном времени, чтобы тренер мог сделать замену, если игрок перегружен. Вне поля важно знать, как игрок спит, ест и что говорят врачи, чтобы планировать тренировочный процесс и предотвращать травмы.

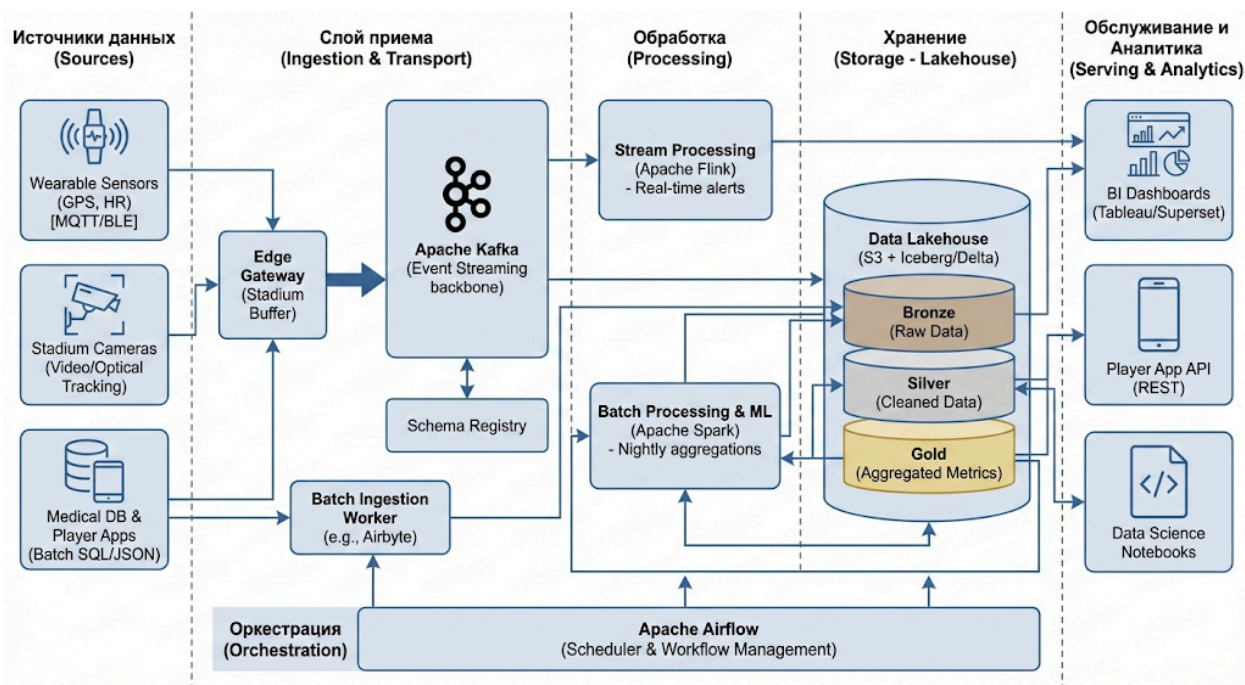


Схема сгенерирована нейросетью по моему примеру и описанию.

Источники данных

Телеметрия с носимых устройств (Wearables & GPS):

- *Что это:* Жилетки с GPS-трекерами и датчики пульса (HRM) на игроках во время тренировок и матчей.
- *Данные:* Координаты (X,Y,Z), скорость, ускорение, ЧСС (частота сердечных сокращений), R-R интервалы (для variability пульса).
- *Формат:* Высокочастотный поток. Компактные бинарные форматы типа Protobuf или сжатый JSON.
- *Объем:* Средний, несколько гигабайт за матч на всю команду.
- *Критичность:* Высокая, данные пульса нужны в реальном времени для безопасности игрока.

Оптический трекинг и видео:

- *Что это:* Камеры, установленные по периметру стадиона.
- *Данные:* Два типа – "сырой" видеопоток и уже обработанные координаты игроков и мяча, которые выдает система компьютерного зрения.
- *Объем:* Видео — огромный (терабайты за матч). Координаты — большой поток структурированных данных.

Медицинские и клубные базы данных:

- *Что это:* Внутренняя CRM/ERP клуба, приложения, где врачи записывают результаты анализов, а игроки отмечают качество сна и настроение.
- *Данные:* Структурированные таблицы (информация об игроке, история травм, информация о крови).
- *Формат:* Реляционные данные в БД.
- *Критичность:* Средняя. Важна точность, но данные обновляются редко (раз в день/неделю).

Слой приёма и передачи данных

Слой отвечает за надежную доставку данных от источника в систему.

Главная проблема — на стадионе может пропасть интернет.

- **Edge Gateway (Шлюз на стадионе):**

- *Как работает:* На стадионе стоит локальный сервер. Все датчики по Bluetooth/ANT+ сначала скидывают данные на него.
- *Надежность:* Gateway имеет локальный буфер. Если связь с облаком пропадает, он накапливает данные у себя, а при появлении связи отправляет всё накопленное. Это гарантирует, что данные о пробеге игрока из-за плохого Wi-Fi не потеряются.

- **Потоковая передача (Streaming Core):**

- *Технология:* Apache Kafka.
- *Как работает:* Gateway отправляет телеметрию и координаты в топик Kafka. Kafka гарантирует, что данные будут сохранены и доступны для обработки, даже если обрабатывающий сервис временно упадет.
- *Схемы данных:* Чтобы не было беспорядка в форматах, используется Schema Registry. Заранее есть договор, например, что сообщение с датчика пульса всегда имеет поля `player_id`, `timestamp`, `heart_rate`. Если датчик пришлёт что-то другое, Kafka это не примет.

- **Пакетная загрузка (Batch Ingestion):**

- *Как работает:* Для медицинских данных не нужен реалтайм. Раз в сутки ночью специальный скрипт подключается к базе данных клиники, забирает новые записи за день и складывает их в хранилище.

Хранение данных

Используется Data Lakehouse. Это значит, что мы храним файлы дешево в облаке, но поверх них используем технологии, которые позволяют работать с ними как с таблицами в базе данных.

- *Технология:* Облачное хранилище объектов (например, S3 или MinIO) в качестве основы. Поверх него используется табличный формат Apache Iceberg.
- *Структура хранения (Медальонная архитектура):*
 - **Bronze (Сырой слой):** Сюда данные падают "как есть". Сырые JSON/Protobuf файлы с датчиков, дампы медицинских баз. Хранится вечно на случай, если нужно будет что-то пересчитать с нуля.
 - **Silver (Очищенный слой):** Здесь лежат данные в формате Parquet/Iceberg. Они уже очищены: дубликаты удалены, временные метки приведены к одному часовому поясу, отфильтрованы явные ошибки датчиков (например, скорость бега 100 км/ч или нулевой пульс во время матча).
 - **Gold (Агрегированный слой):** Готовые витрины данных для аналитиков и тренеров. Например, таблица "Итоги дня по игроку", где одной строкой записано: сколько пробежал, средний пульс, оценка сна, вердикт врача.

Слой обработки и преобразования данных

Здесь происходит превращение сырых цифр в полезные инсайты.

- **Потоковая обработка (Real-time / Stream Processing):**
 - *Технология:* Apache Flink или Kafka Streams.
 - *Зачем:* Для оперативного реагирования во время матча.
 - *Что делает:* Flink читает поток из Kafka `live_sensor_data`. Он сразу считает скользящие средние. Например: "Если средний пульс игрока X за последние 3 минуты выше 180 уд/мин — сгенерировать алерт". Этот алерт сразу идёт в отдельный топик Kafka, который слушает планшет тренера.
- **Пакетная обработка (Batch Processing):**

- *Технология:* Apache Spark.
- *Зачем:* Для глубокого анализа после матча и в конце дня.
- *Что делает (ETL):* Каждую ночь Spark запускается, берет сырые данные (Bronze) за прошедшие сутки, очищает их и складывает в Silver слой. Затем он берет данные из Silver, объединяет GPS-треки с медицинскими данными и считает сложные метрики (например, "индекс усталости", который зависит и от нагрузки на тренировке, и от качества сна). Результат кладется в Gold слой.
- *ML модели:* Также здесь применяются модели машинного обучения. Например, модель предсказания риска травмы, которая раз в сутки оценивает состояние игрока на основе всех накопленных исторических данных.

Слой обслуживания данных и аналитики

Получение результатов конечными пользователями.

- **BI Дашборды:**

- *Технология:* Tableau / Power BI / Apache Superset.
- *Как работает:* Подключаются напрямую к Gold слою нашего Lakehouse (через SQL-интерфейс, например, Trino или Athena) и рисуют информативные графики для тренерского штаба: динамика формы игрока за месяц, сравнение игроков и другие.

- **API для приложений:**

- Для мобильного приложения игрока (где он смотрит свою статистику) или болельщика есть REST API сервис. Он быстро читает готовые агрегаты из Gold слоя (или из базы данных типа Redis/Postgres, куда данные дублируются для скорости) и отдает их приложению.

- **Каталог данных и Безопасность:**

- Чтобы аналитики не запутались в таблицах, используем Data Catalog (например, DataHub). Он показывает, где какие данные лежат и откуда

они взялись.

- *Доступ:* Врачи видят медицинские данные, тренеры — только спортивные показатели. Это реализуется через ролевую модель доступа на уровне слоя обслуживания.

Оркестрация

Чтобы всё запускалось вовремя и в нужном порядке

- *Технология:* Apache Airflow.
- *Как работает:* В Airflow описаны DAG (направленные ациклические графы задач).
- *Пример DAG "Ночной расчёт":*
 1. В 2 часа ночи запустить задачу "Импорт медицинских данных из клиники".
 2. Только после успешного завершения шага 1 и проверки, что все данные с датчиков за день доехали до S3, запустить Spark-задачу "Расчет дневных агрегатов (Silver → Gold)".
 3. Если Spark-задача упала с ошибкой, попробовать перезапустить её 3 раза, а если не вышло — отправить уведомление дежурному дата-инженеру в Slack.
 4. После успешного расчета запустить задачу "Обновление ML-модели риска травм".